Multi – Level linked list

–Insert At Position

–Program Analysis and Time Complexity

Program:

```
void insertAtPosition()
{
    int data, pos;
    cout << "Enter data: ";
    cin >> data;
    cout << "Enter position (1-based): ";
    cin >> pos;

    Node *newNode = createNode(data);

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of range." << endl;
        free(newNode);
        return;
    }
```

```
    else
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted at position " << pos << endl;
    }
}
……
int main()
{
    int choice;
    while (true)
    {
      cout << "3. Insert at Position (Main)" << endl;
       cin >> choice;
        switch (choice)
        {
        case 3:
            insertAtPosition();
            break;
       default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Program Analysis

This program counts from index $\mathbf{1}$ ***based i.e.***

$[\mathbf{1}, \mathbf{2}, \mathbf{3}, \ldots . n \textit{ not } \mathbf{0}, \mathbf{1}, \mathbf{2}, \ldots, n-\mathbf{1}]$

Say, $int\ data = \mathbf{20}$.

Position : $\mathbf{1}$.

$Node * newNode = createNode(data: \mathbf{20});$

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
```

Next,

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ ***The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.***

→ ***The size of a pointer*** $(newNode)$ ***is typically*** $\mathbf{8}$ ***bytes on most modern*** $\mathbf{64}$

$-$***bit systems, regardless of the size of the structure it points to.***

→ ***The size of the Node structure itself is determined by the sum of the sizes of its members***:

→ ***data*** **(*an integer*)** ***is typically*** **4** ***bytes.***

→ ***next*** **(*a pointer pointing to another Node*)** ***is typically*** **8** ***bytes.***

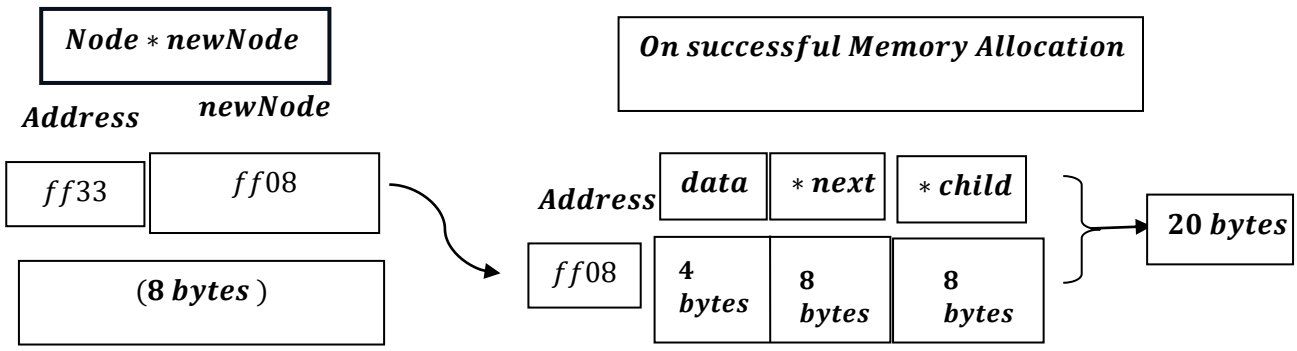
→ ***child*** **(*a pointer pointing to child node*)** ***is typically*** **8** ***bytes.***

→ ***Therefore , the size of the Node structure is typically*** **20** ***bytes*** **(** **4** ***bytes for data*** **+** **8** ***bytes for next*** **+** **8** ***bytes for child*** **)** .

What does it mean?

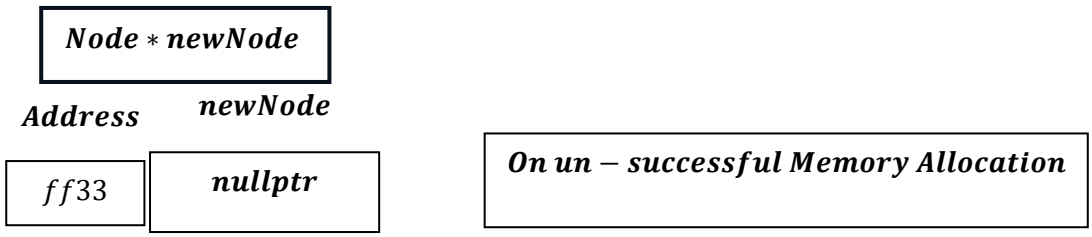
→ ***newNode will try to allocate memory for integer data for*** **4** ***bytes and next pointer variable i.e. typically*** **8** ***bytes, also child next pointer variable also typically*** **8** ***bytes hence assumed total space consumed is total*** **(4 + 8 + 8)*bytes* = 20 *bytes*.**

Say,



But on Un − Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



Now, if the value of the pointer variable newNode is nullptr, then:

One may output : `Overflow ***or Memory Allocation Failed as a message.***`

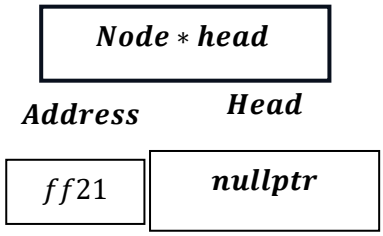
Now,

```
      case 3:
            insertAtPosition();
            break;
```

Now , this will mainly create Main node not a child node.

How?

```
Node *head =nullptr
```



And our data input is : 20.

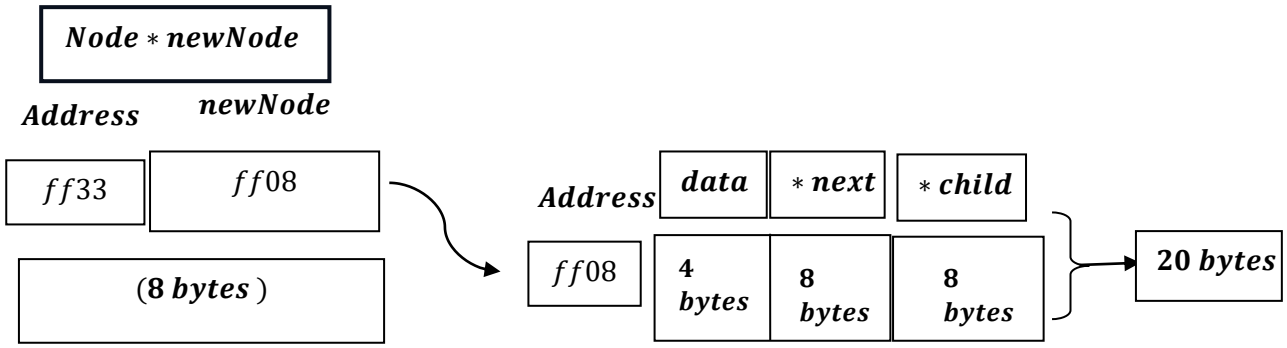
```
Node *createNode(int data){…}
```

Return type is struct type pointer variable as mentioned here:

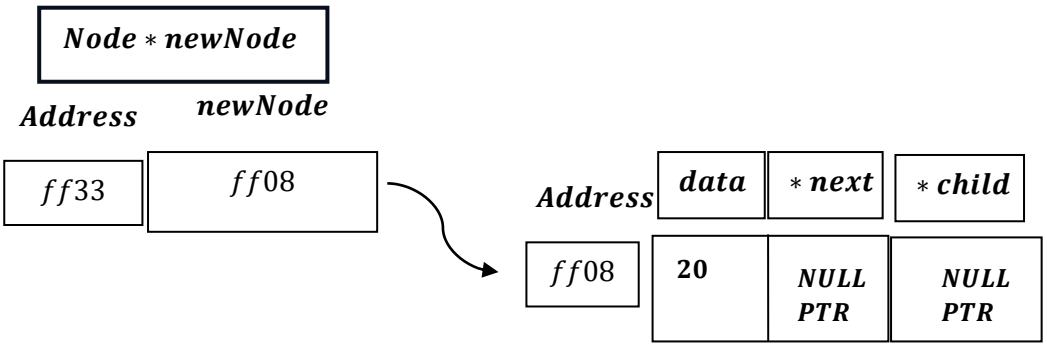
Node * and function name is createNode(variable : 20).

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

We got:



```
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
```




```
    return newNode;
```

As we already know , Return type is struct type pointer variable . Return newNode will return the value stored inside newNode, pointer :

i.e. return newNode : ff08 .

Now,

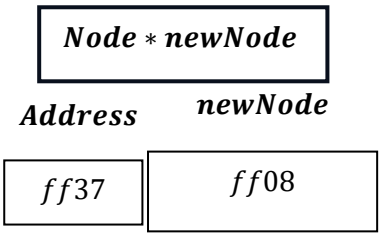
```
    Node *newNode = createNode(data);
```

Node * newNode = ff08.

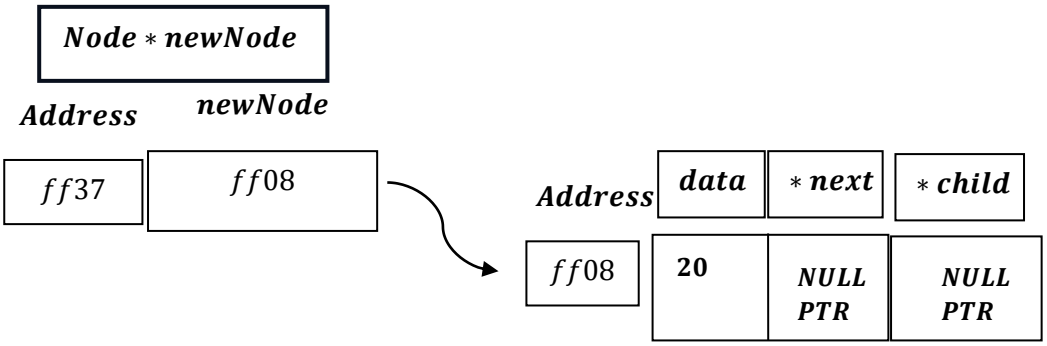
This newNode is another pointer variable , under scope of insertAtBeginning() function, not to be mixed with createNode() function .

For * newNode of insertAtPosition() function:

Node * newNode = ff08.



Now pointer * newNode connecting to address ff08.




```
    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }
```

$$if\ Pos = 1:$$

$$newNode \rightarrow next = head;$$

$$Now, we\ have\ to\ remember\ that\ this\ works\ for:$$

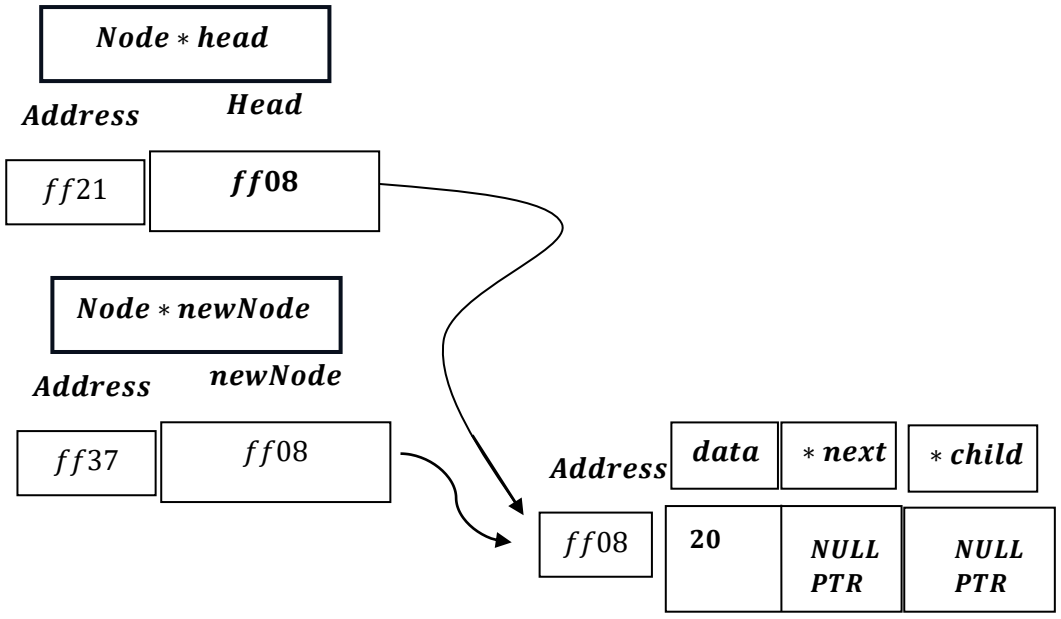
$$1) When\ List\ is\ Empty$$

$$2) When\ List\ is\ not\ Empty$$

$$As\ for\ ListIsEmpty, this\ just\ assign\ same\ thing\ i.e.\ NULLPTR:$$

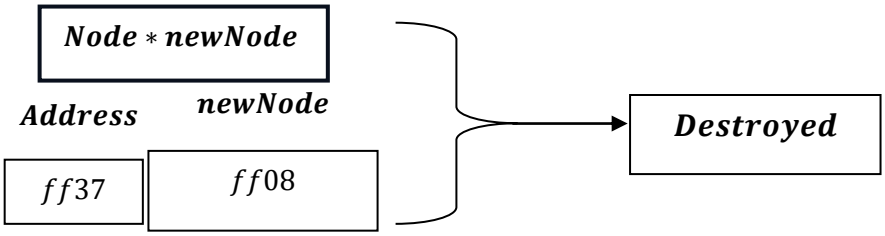
$$newNode \rightarrow Next = Head = NULLPTR$$

$$Head = newNode = ff08.$$

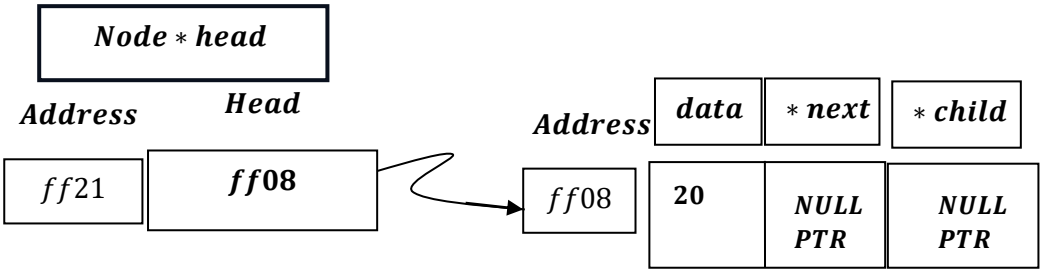


Return statement, will return void and will exit from the program.

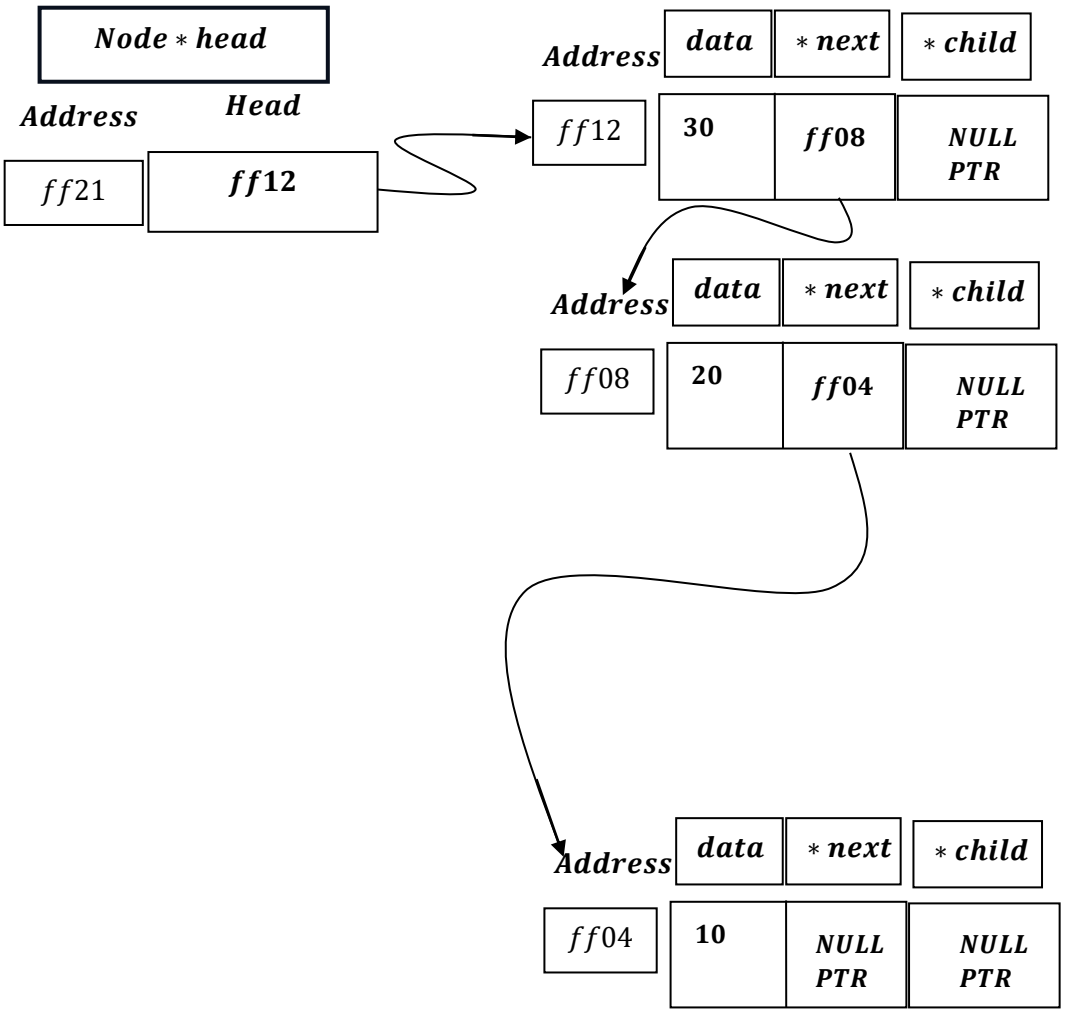
And , Node * newNode of insertAtPosition(), gets destroyed after execution of the function:



And we are left with:



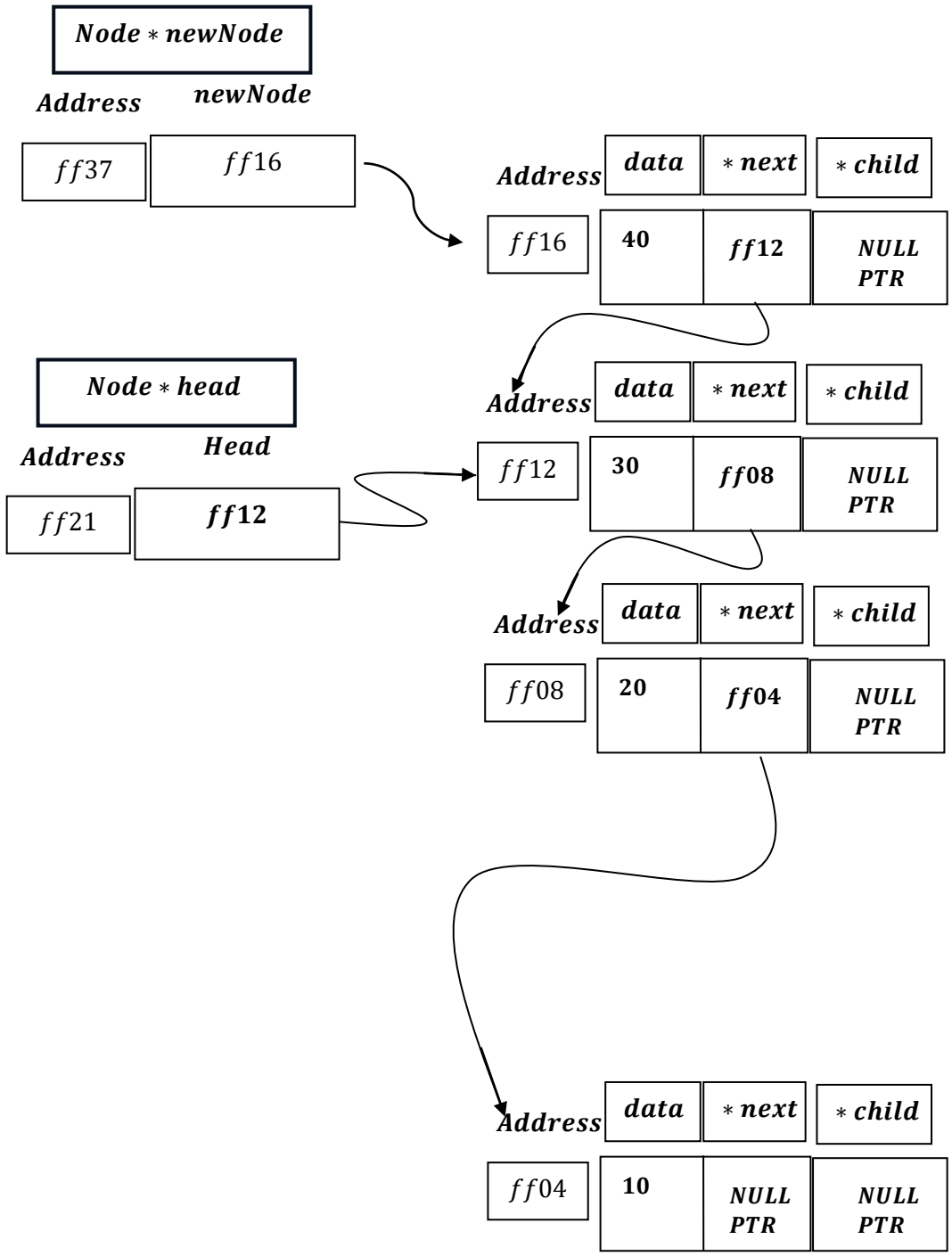
List Is Not Empty



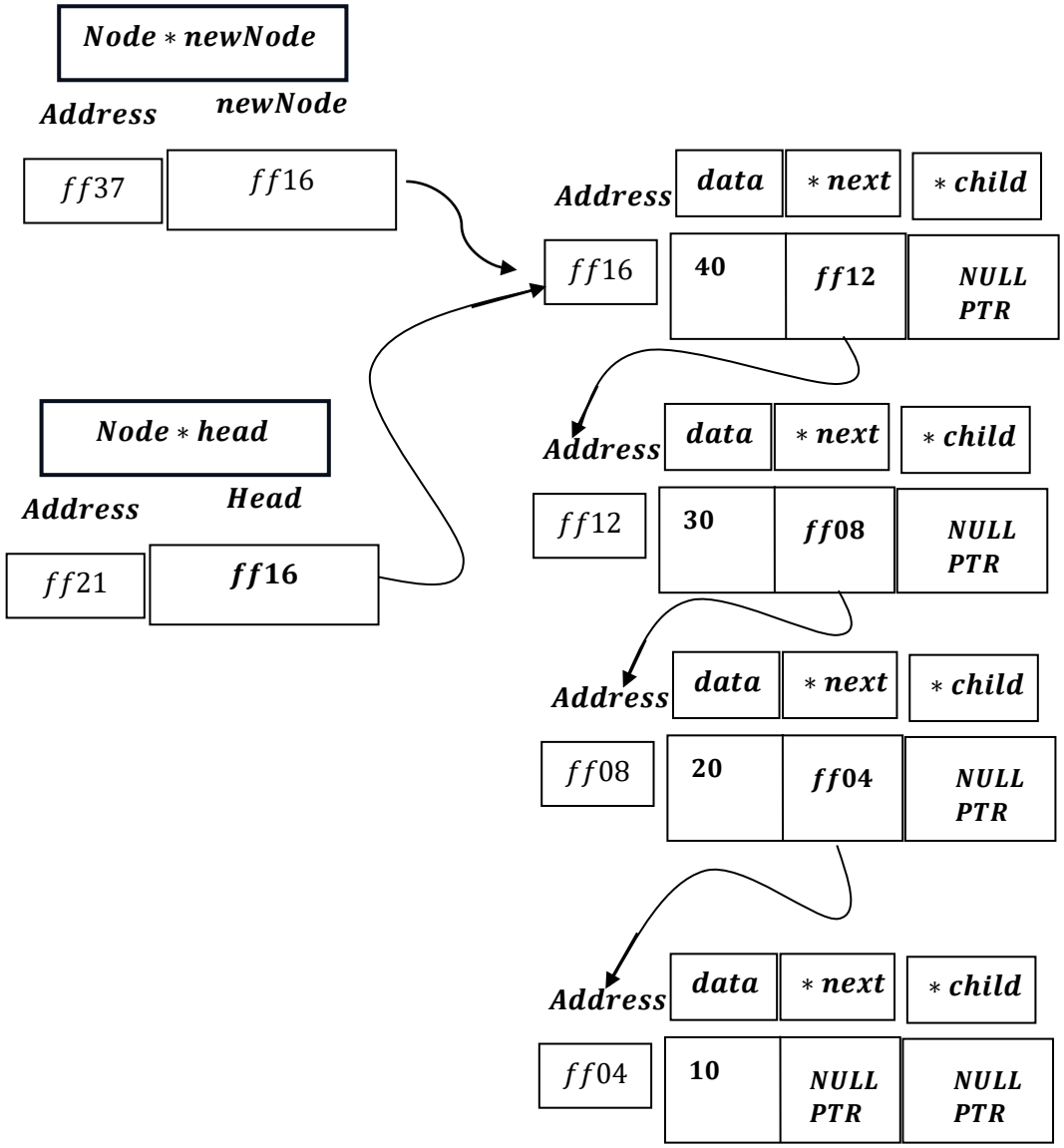
Here, if pos = 1.

```
    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }
```

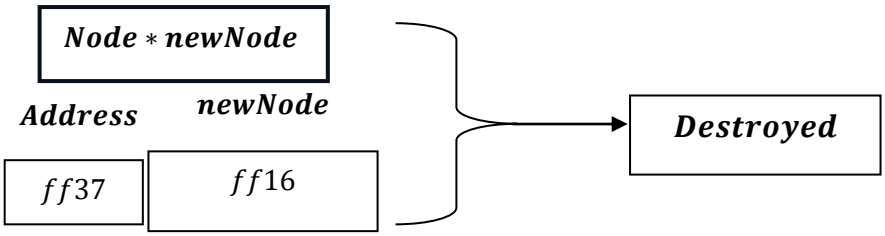
$$newNode \rightarrow Next = Head = ff12.$$



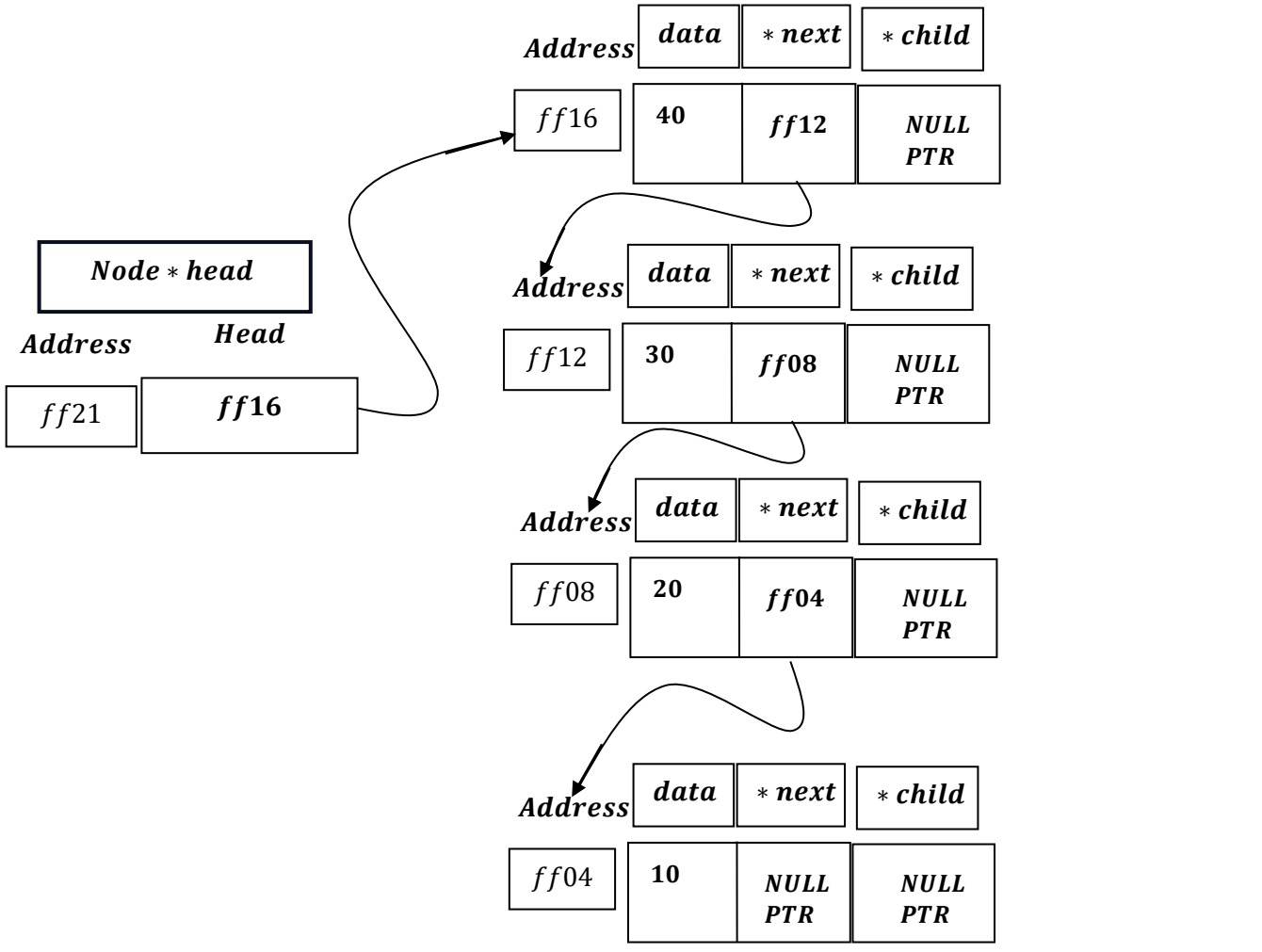
$\boldsymbol{Head = newNode = ff16}$.



And , Node * newNode of insertAtPosition(), gets destroyed after execution of the function:



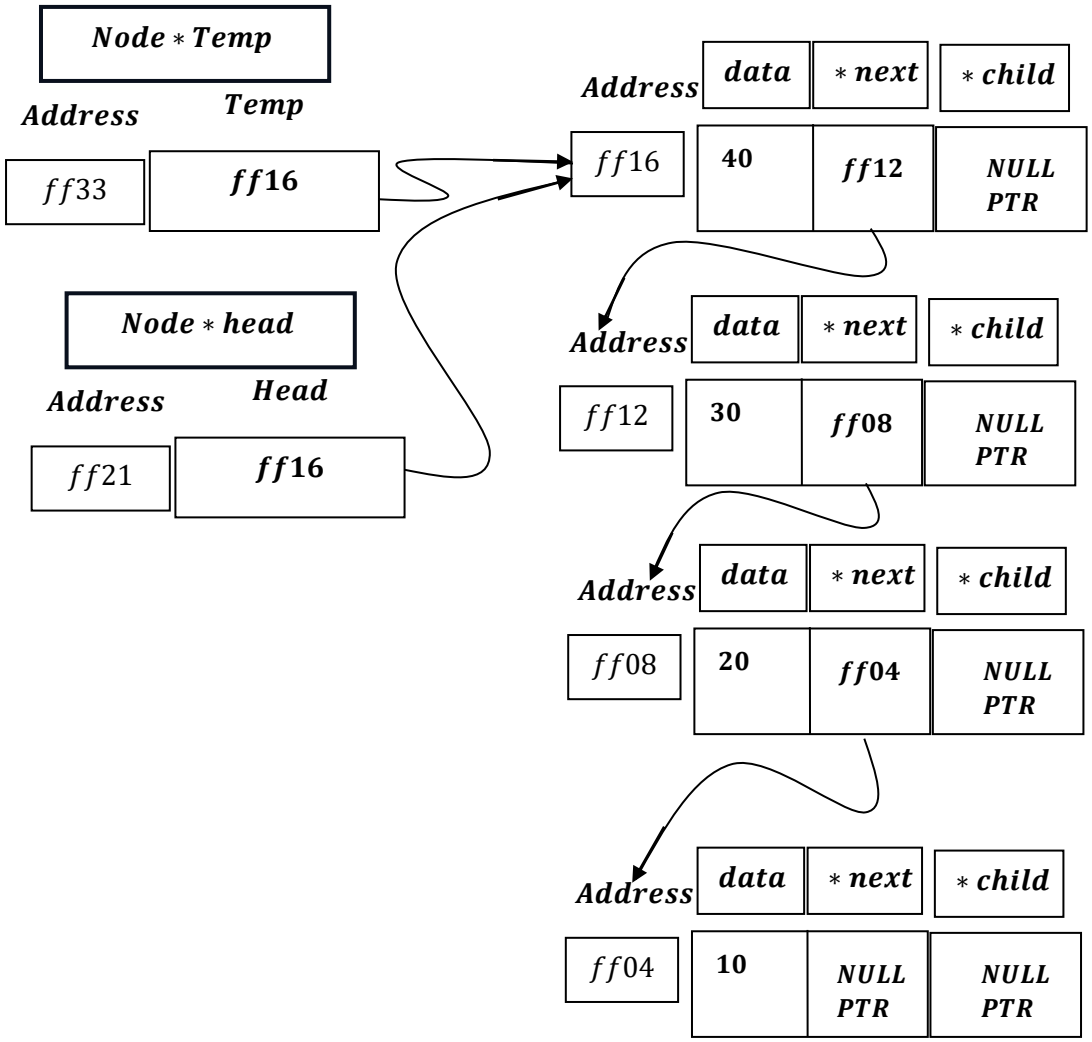
And we are left with:



If we want to enter data = 05 at Pos = 5.

```
    Node *temp = head;
```

$$TEMP = HEAD = ff16$$



```
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }
```

$$\boldsymbol{Here\ ,i\ will\ go\ from\ 1\ to\ 1\ less\ than\ pos-1\ nodes\ and\ temp \neq NULLPTR}\begin{bmatrix}\boldsymbol{Not}\\ \boldsymbol{Out\ of\ Bound}\end{bmatrix}:$$

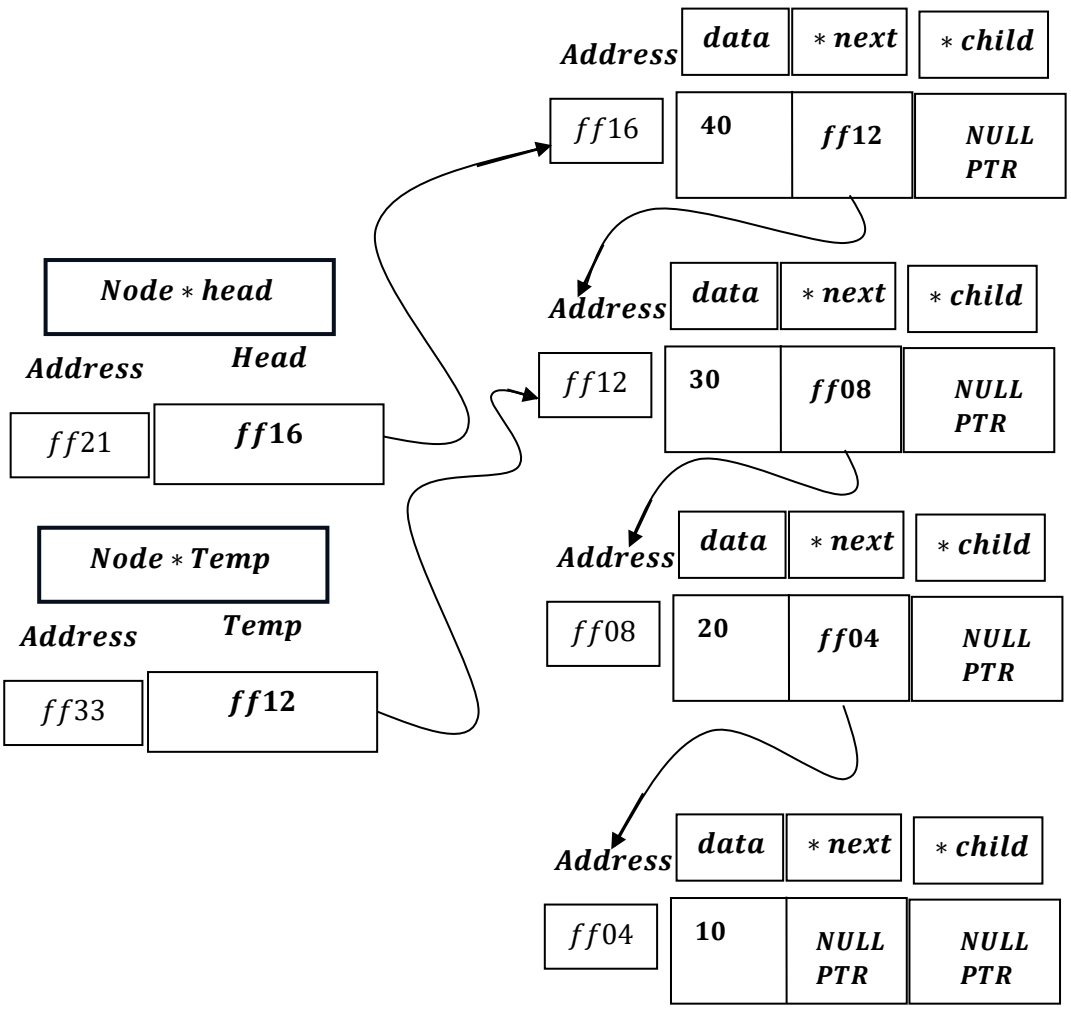
$$\boldsymbol{Here, we\ want\ to\ enter\ at\ POS = 5\ , hence\ it\ will\ go\ from\ 1\ to\ 5-2=3\ times\ iteration.}$$

$$\underline{\boldsymbol{[Loop]}}$$

$$\boldsymbol{When\ LoWER\ Bound: i = 1\ ; Upper\ Bound: i < 4[i = 3];}$$

$$\boldsymbol{And\ Temp \neq NULLPTR: (Condition\ \ is\ True)}$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = ff12}$$

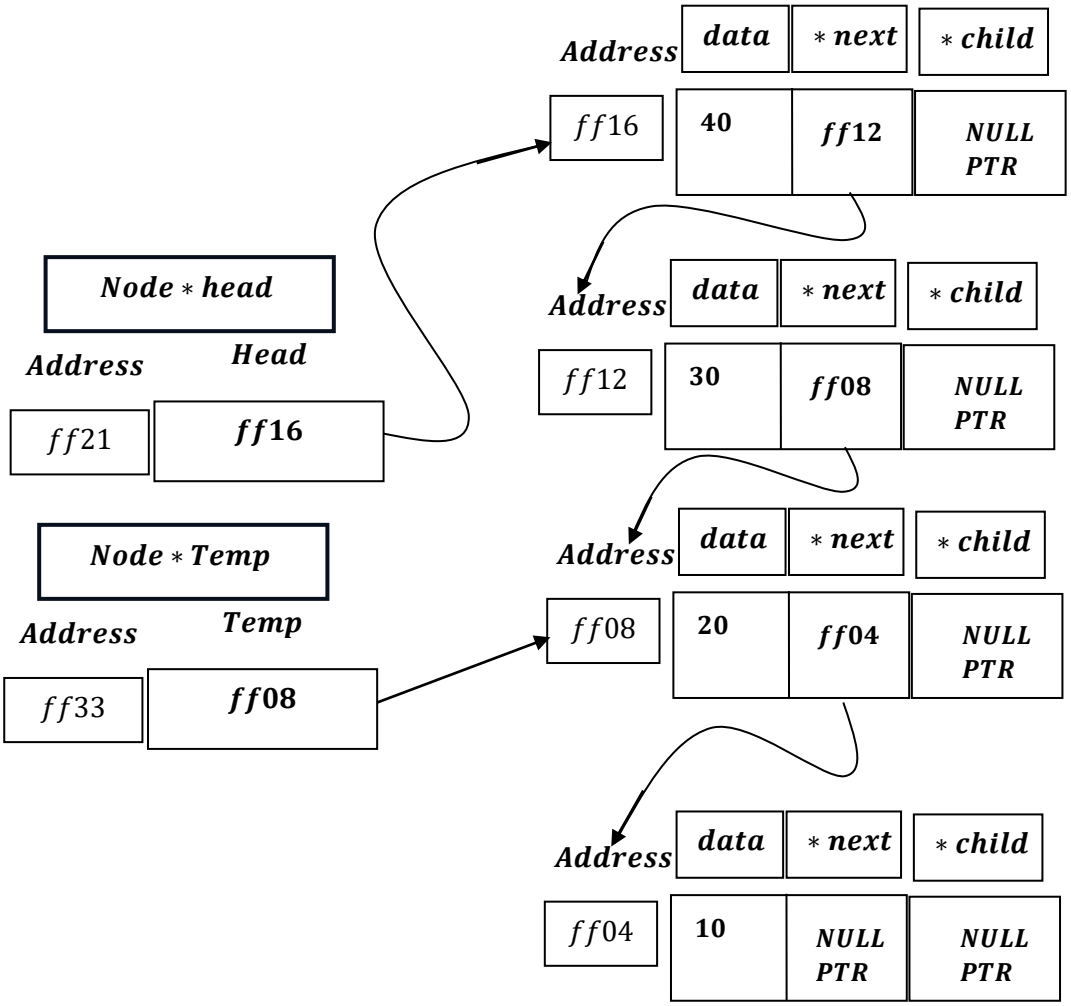


$$\textbf{\textit{Update of Loop}}\ (\textbf{\textit{Post Operation}}): i = i + 1 = 1 + 1 = 2.$$

$$\boldsymbol{When\ LoWER\ Bound}: \boldsymbol{i = 2}\ ; \boldsymbol{Upper\ Bound}: \boldsymbol{i < 4}[\boldsymbol{i = 3}];$$

$$\boldsymbol{And\ Temp \neq NULLPTR}: (\boldsymbol{Condition\ \ is\ True})$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = ff08}$$

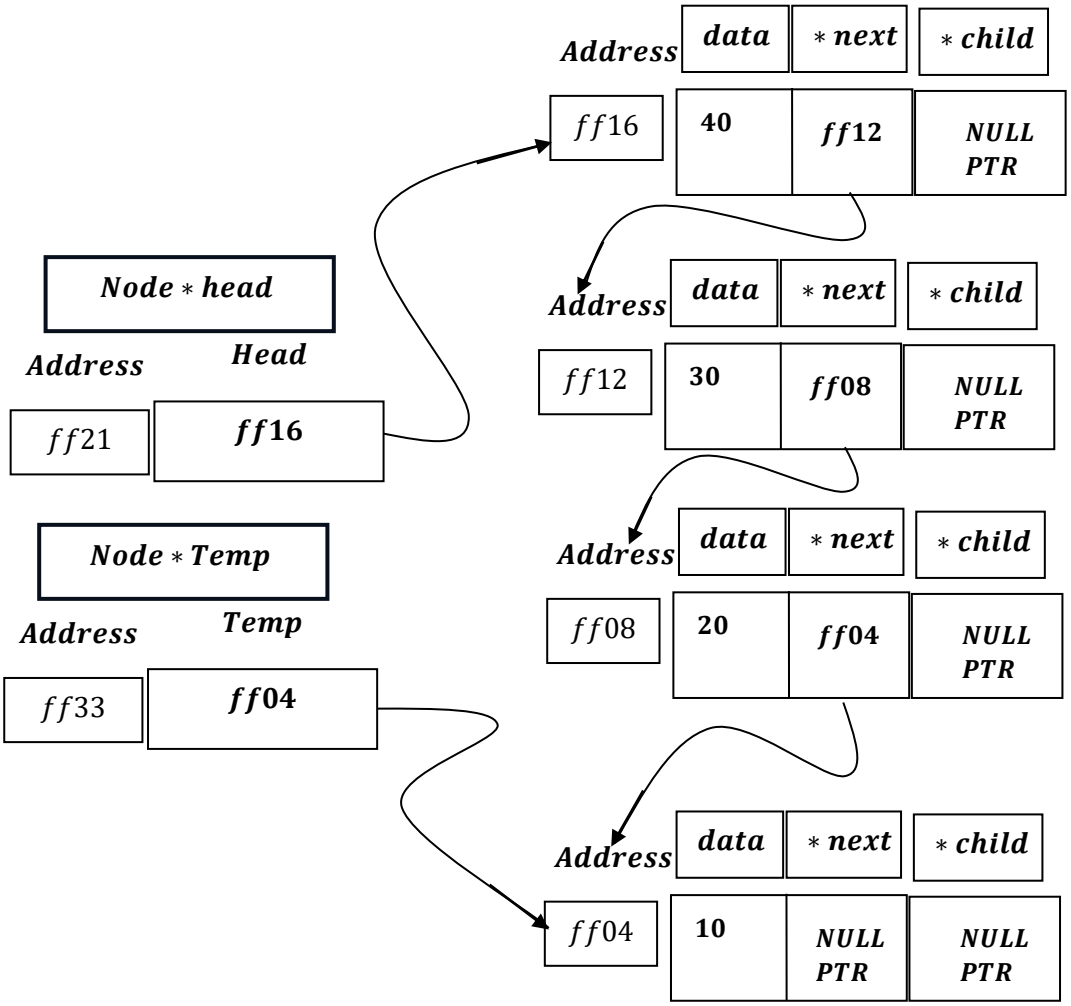


$$\boldsymbol{Update\ of\ Loop\ (Post\ Operation)}: \boldsymbol{i = i + 1 = 2 + 1 = 3}.$$

$$\boldsymbol{When\ LoWER\ Bound}: \boldsymbol{i = 3}\ ; \boldsymbol{Upper\ Bound}: \boldsymbol{i < 4}[\boldsymbol{i = 3}];$$

$$\boldsymbol{And\ Temp \neq NULLPTR}: (\boldsymbol{Condition\ \ is\ True})$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = ff04}$$



$$\boldsymbol{Update\ of\ Loop\ (Post\ Operation)}: \boldsymbol{i = i + 1 = 3 + 1 = 4}.$$

$$When\ LoWER\ Bound: i = 4\ ; Upper\ Bound: i < 4[i = 3];$$

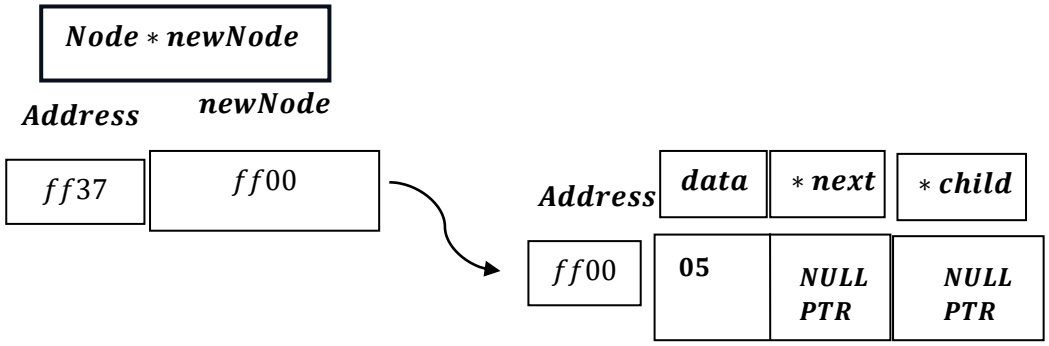
$$And\ Temp \neq NULLPTR: \begin{pmatrix} Condition\ \ is\ False\ as \\ Lower\ Bound > Upper\ Bound \end{pmatrix}$$

```
    if (temp == nullptr)
    {
        cout << "Position out of range." << endl;
        free(newNode);
        return;
    }
    else
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted at position " << pos << endl;
    }
```

$$As\ Temp\ \neq NULLPTR\ , and\ ``NOT\ OUT\ OF\ RANGE``,$$

$$Therefore\ it\ will\ run\ else\ part:$$

```
    else
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted at position " << pos << endl;
    }
```

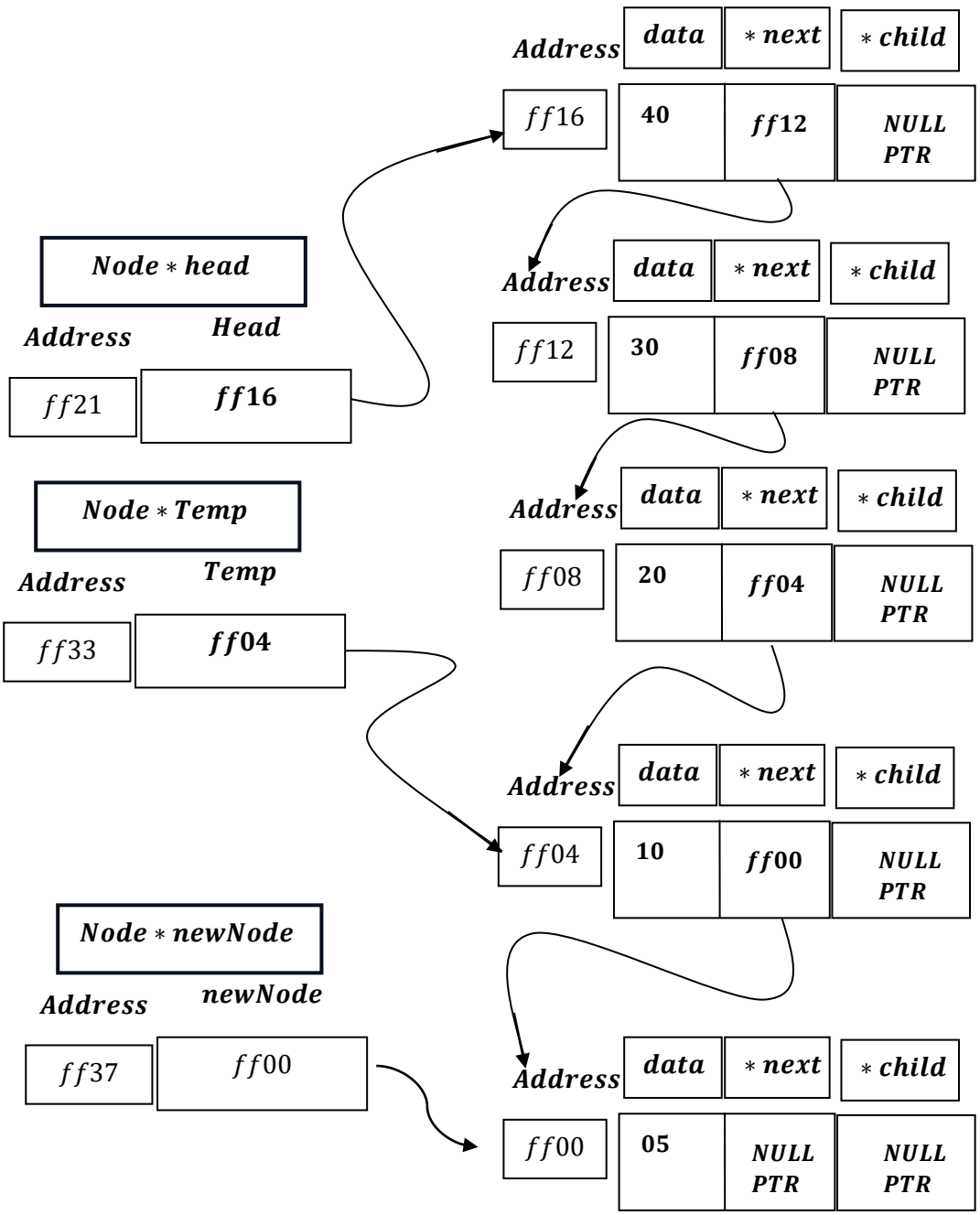



```
        newNode->next = temp->next;
```

$$newNode \rightarrow Next = temp \rightarrow next = NULLPTR \begin{bmatrix} No\ Change \\ ,if\ it\ was\ not \\ entered\ as \\ last\ node, \\ then\ there \\ can\ be\ a \\ change\ . \end{bmatrix}$$

```
        temp->next = newNode;
```

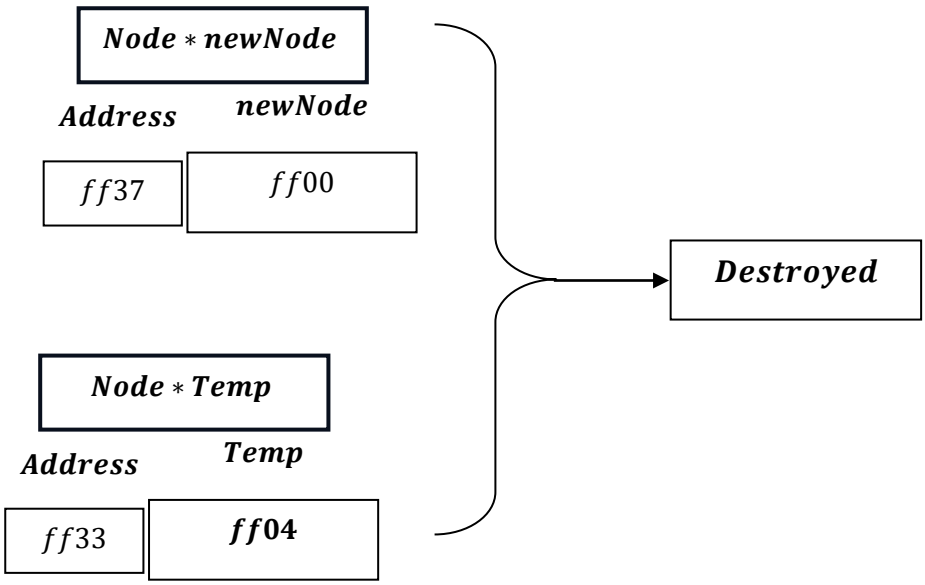
$Temp \rightarrow next = newNode = ff00.$


Address
data
* next
* child
ff16
40
ff12
NULL PTR
Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Node * Temp
Address
Temp
ff33
ff04
Address
data
* next
* child
ff08
20
ff04
NULL PTR
Address
data
* next
* child
ff04
10
ff00
NULL PTR
Node * newNode
Address
newNode
ff37
ff00
Address
data
* next
* child
ff00
05
NULL PTR
NULL PTR

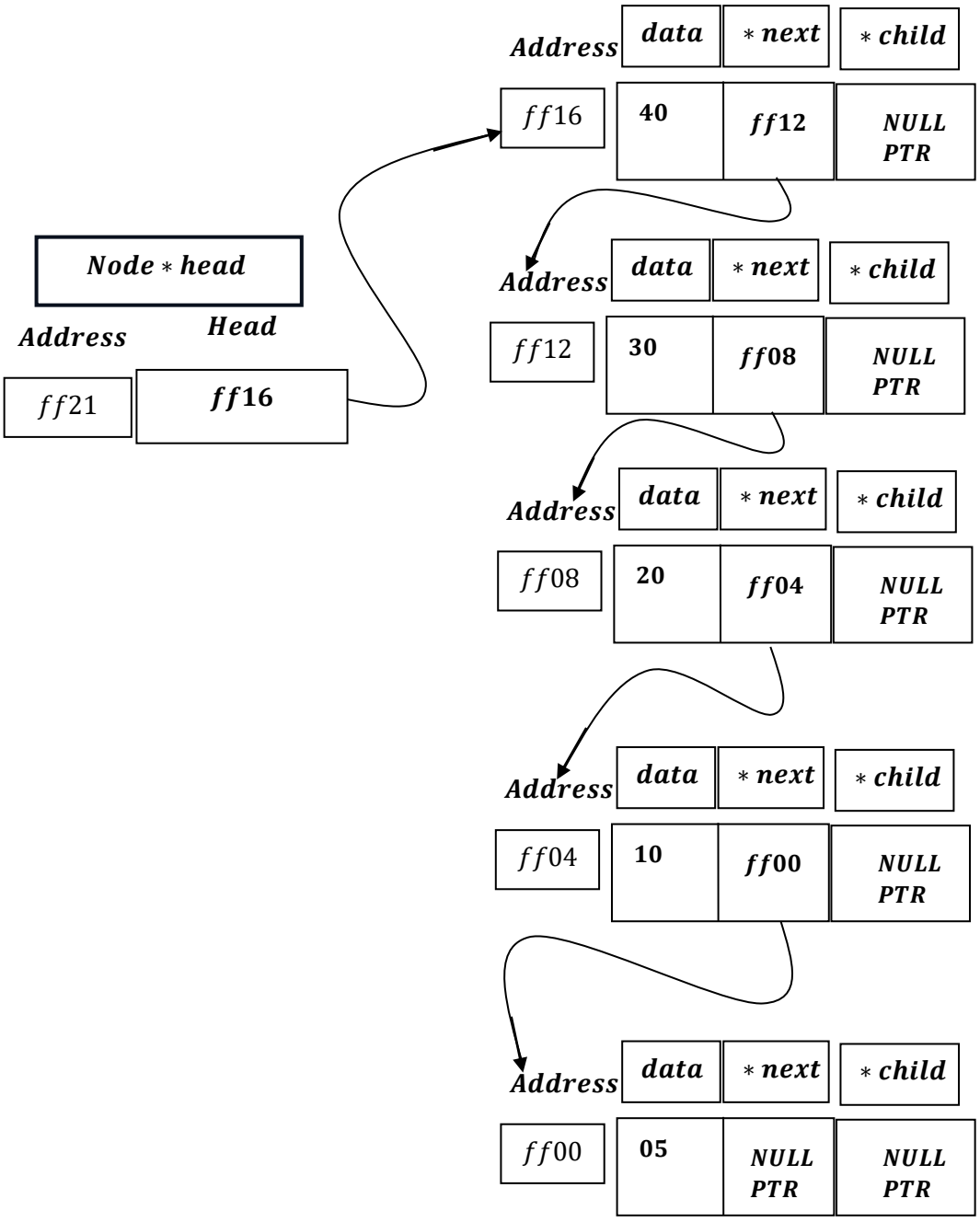
Next, It will print :

``***Inserted at position***:`` , **5**.

And , Node * newNode and Node * Temp of insertAtPosition(),
gets destroyed after execution of the function:



And , we are left with:

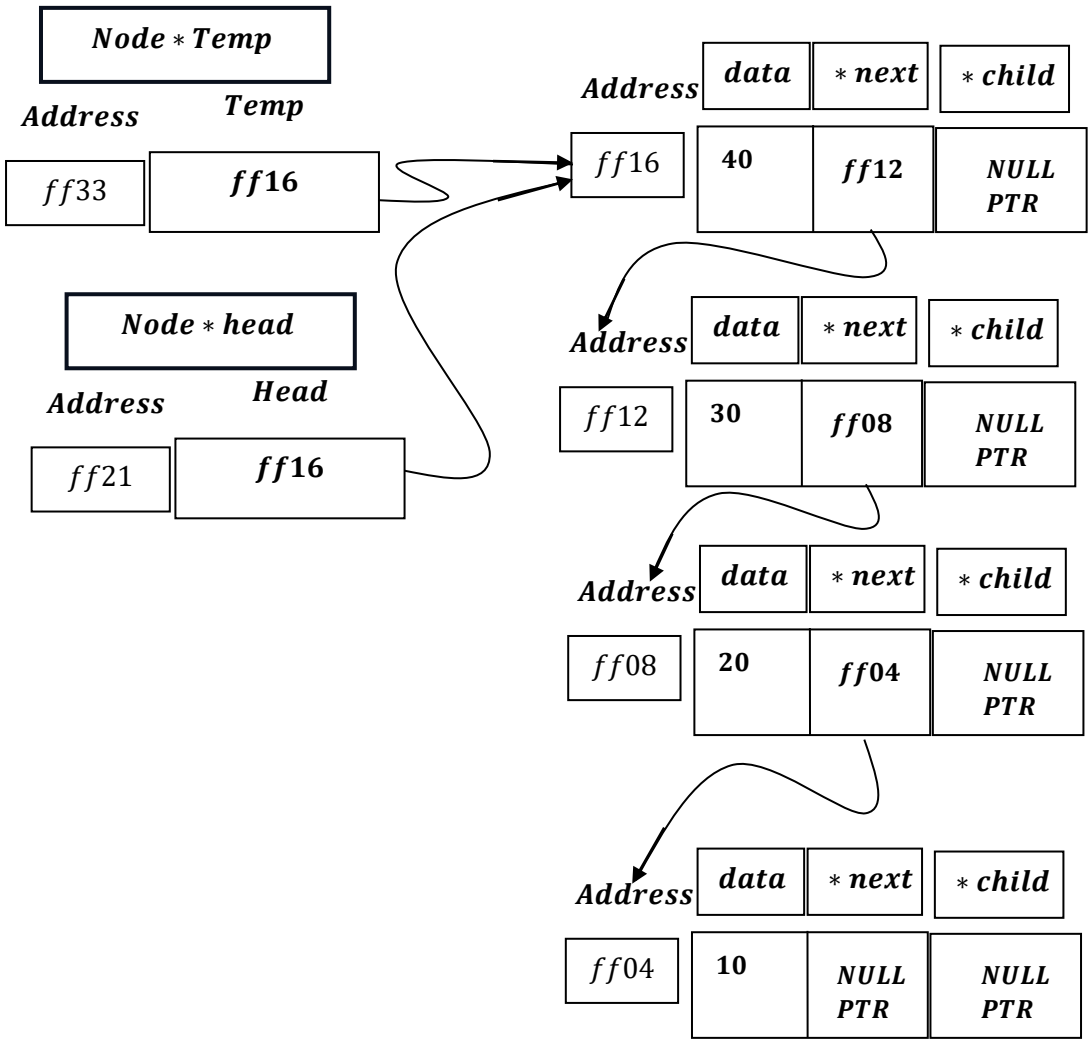

Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff16
40
ff12
NULL PTR
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
NULL PTR
Address
data
* next
* child
ff04
10
ff00
NULL PTR
Address
data
* next
* child
ff00
05
NULL PTR
NULL PTR

Now, Out of Range:

If we want to enter data = 05 at Pos = 6.

```
    Node *temp = head;
```

$$TEMP = HEAD = ff16$$



```
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }
```

$\boldsymbol{Here\ ,i\ will\ go\ from\ 1\ to\ 1\ less\ than\ pos-1\ nodes\ and\ temp \neq NULLPTR:}$

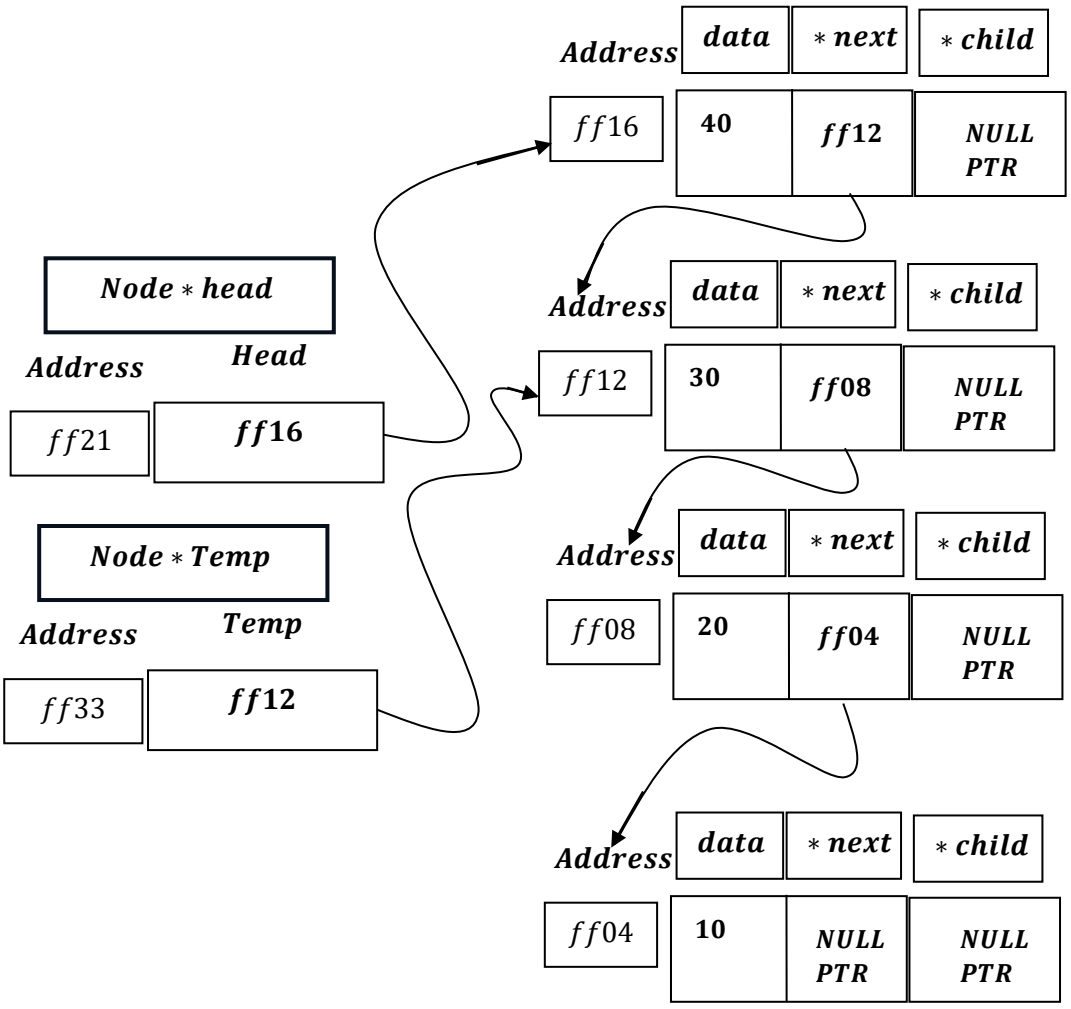
$\boldsymbol{Here, we\ want\ to\ enter\ at\ POS = 6\ , hence\ it\ will\ go\ from\ 1\ to\ 6-2=4\ times\ iteration.}$

$\underline{\boldsymbol{[Loop]}}$

$\boldsymbol{When\ LoWER\ Bound: i = 1\ ; Upper\ Bound: i < 5[i = 4];}$

$\boldsymbol{And\ Temp \neq NULLPTR: (Condition\ \ is\ True)}$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = ff12}$$

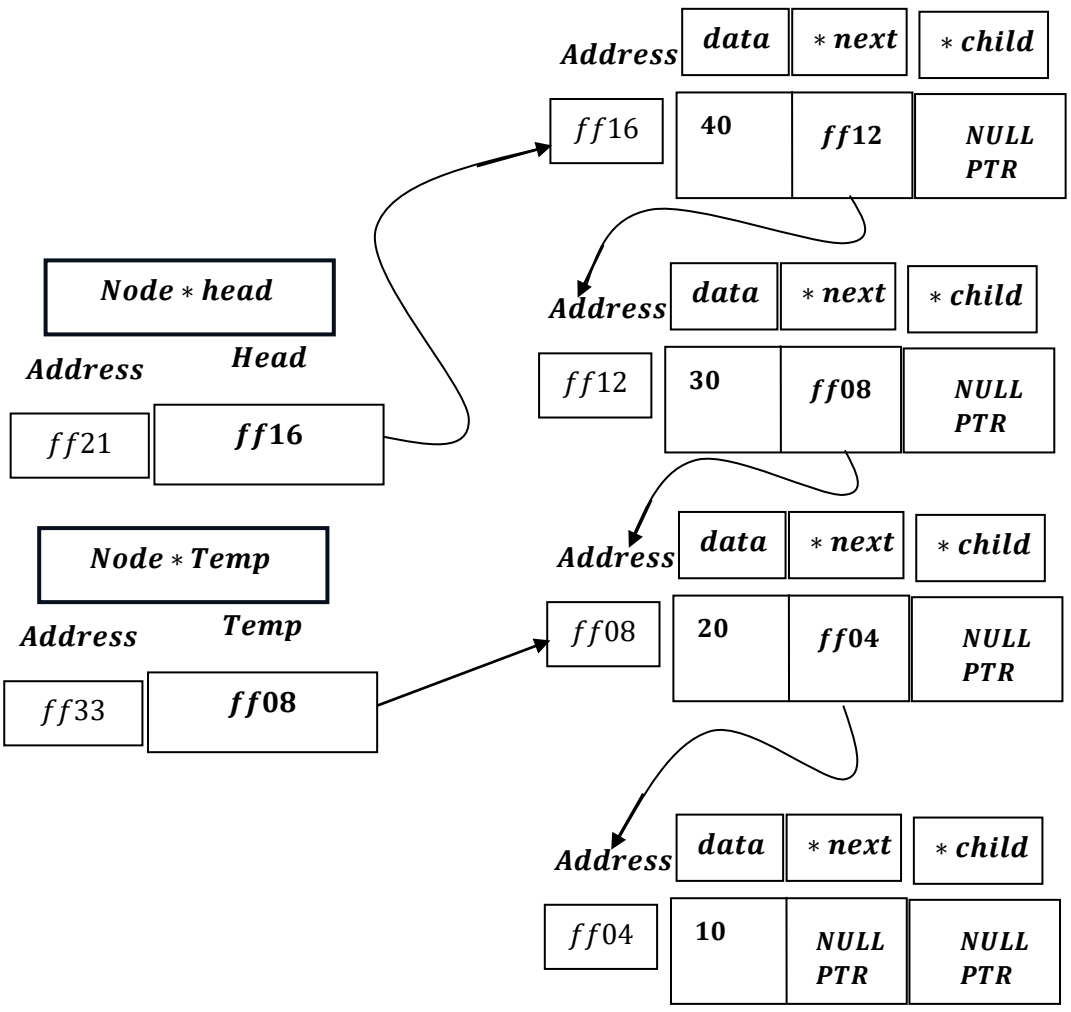


***Update of Loop (Post Operation)*: $i = i + 1 = 1 + 1 = 2$.**

$$\boldsymbol{When\ LoWER\ Bound}: \boldsymbol{i = 2}\ ; \boldsymbol{Upper\ Bound}: \boldsymbol{i < 5[i = 4]};$$

$$\boldsymbol{And\ Temp \neq NULLPTR}: (\boldsymbol{Condition\ \ is\ True})$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = ff08}$$

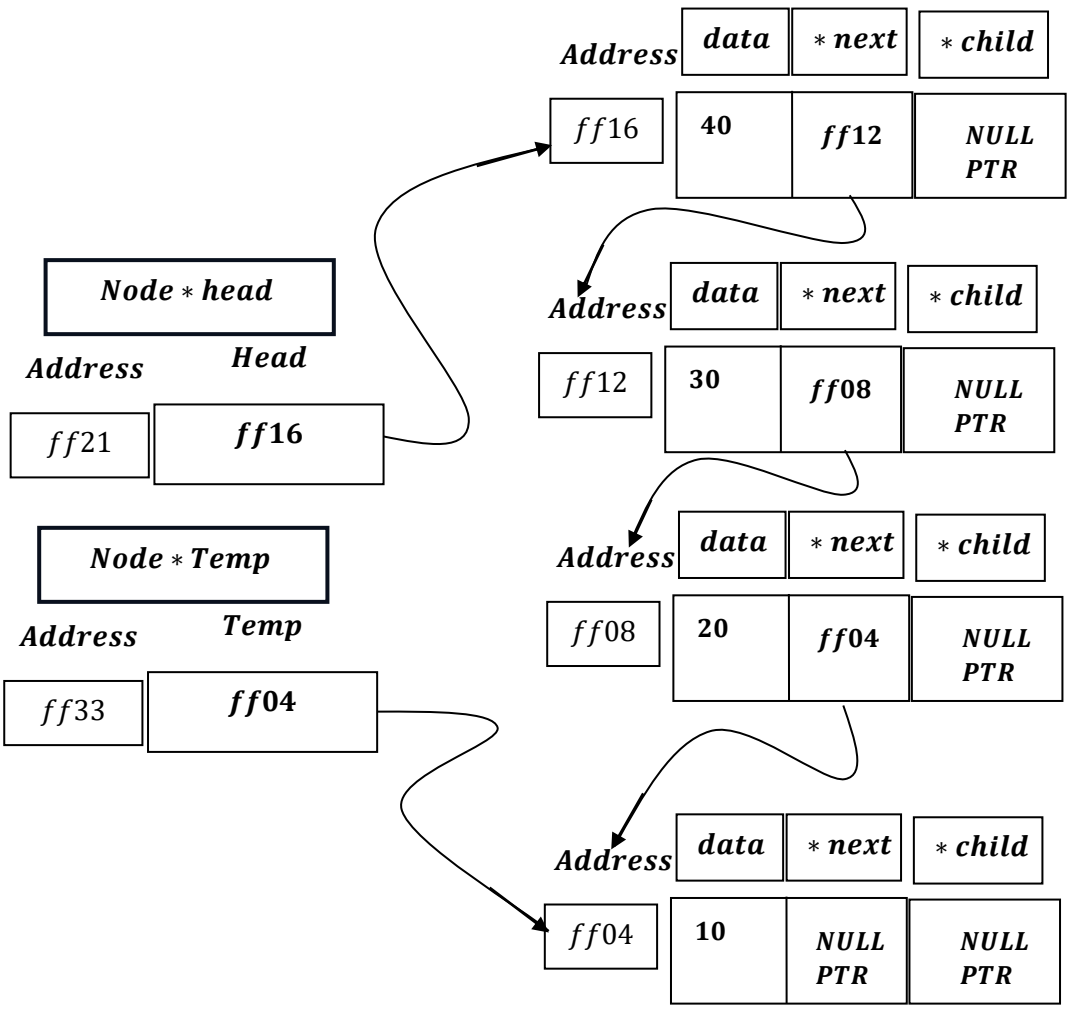


$$\boldsymbol{Update\ of\ Loop\ (Post\ Operation): i = i + 1 = 2 + 1 = 3.}$$

$$\boldsymbol{When\ LoWER\ Bound: i = 3\ ; Upper\ Bound: i < 5[i = 4];}$$

$$\boldsymbol{And\ Temp \neq NULLPTR: (Condition\ \ is\ True)}$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = ff04}$$

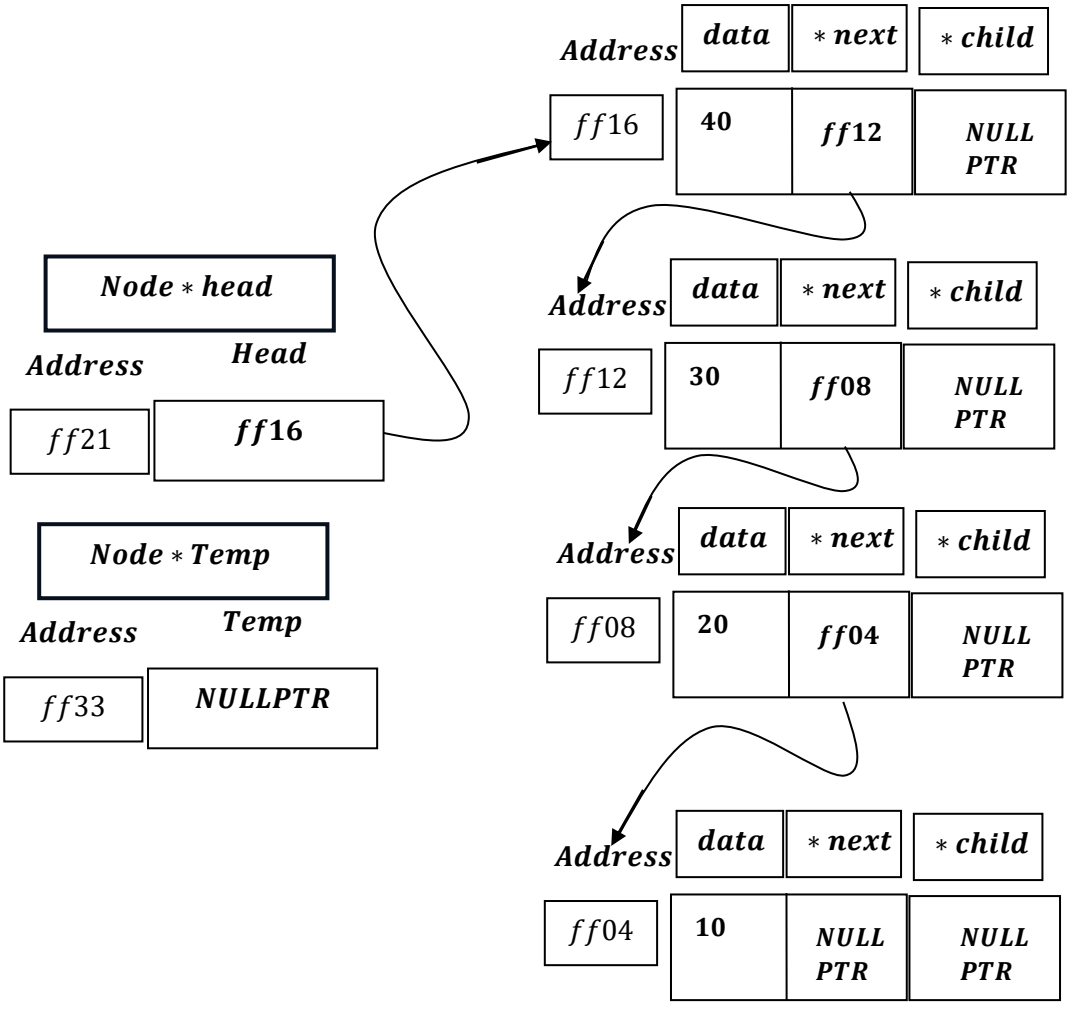


$$\boldsymbol{Update\ of\ Loop\ (Post\ Operation): i = i + 1 = 3 + 1 = 4.}$$

$$\boldsymbol{When\ LoWER\ Bound: i = 4\ ; Upper\ Bound: i < 5[i = 4];}$$

$$\boldsymbol{And\ Temp \neq NULLPTR: (Condition\ \ is\ True)}$$

$$\boldsymbol{TEMP = TEMP \rightarrow NEXT = NULLPTR}$$



$$\boldsymbol{Update\ of\ Loop\ (Post\ Operation): i = i + 1 = 4 + 1 = 5.}$$

$$When\ LoWER\ Bound: i = 5\ ; Upper\ Bound: i < 5[i = 4];$$

$$And\ Temp \neq NULLPTR: \begin{pmatrix} Condition\ \ is\ False, as, \\ Lower\ Bound > Upper\ Bound \\ And \\ Temp = NULLPTR. \end{pmatrix}$$

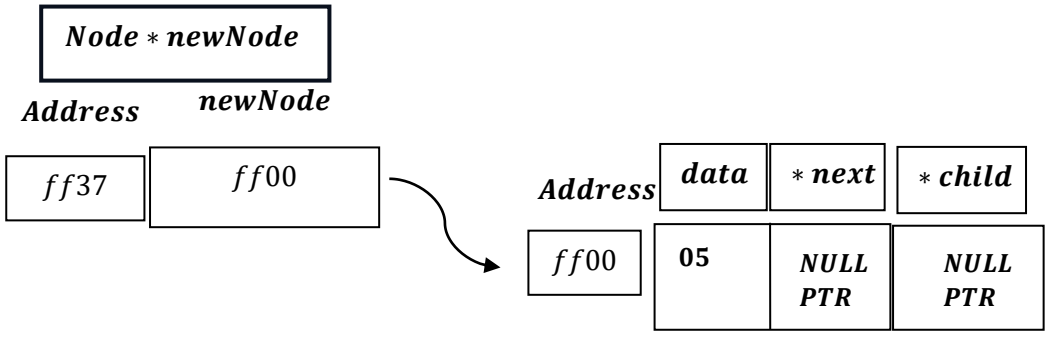
$$Hence\ Loop\ Exits.$$

```
    if (temp == nullptr)
    {
        cout << "Position out of range." << endl;
        free(newNode);
        return;
    }
```

$$As\ Temp = NULLPTR:$$

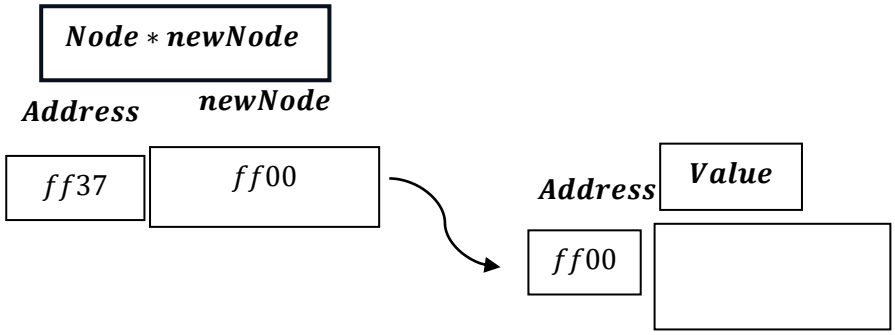
$$It\ will\ print\ ``Position\ out\ of\ range.``$$

$$As\ , the\ new\ \ node\ was\ created\ earlier\ , hence,$$

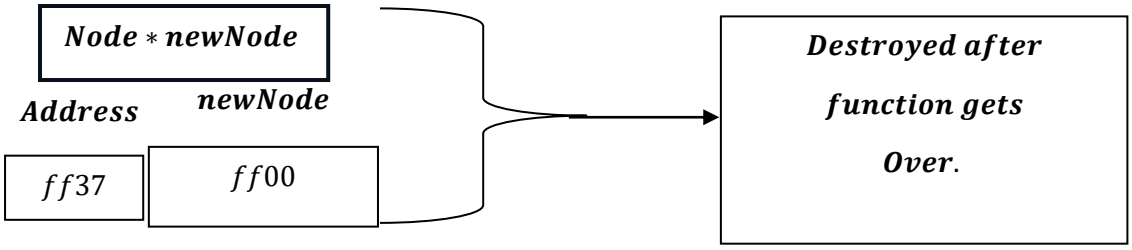



```
        free(newNode);
```

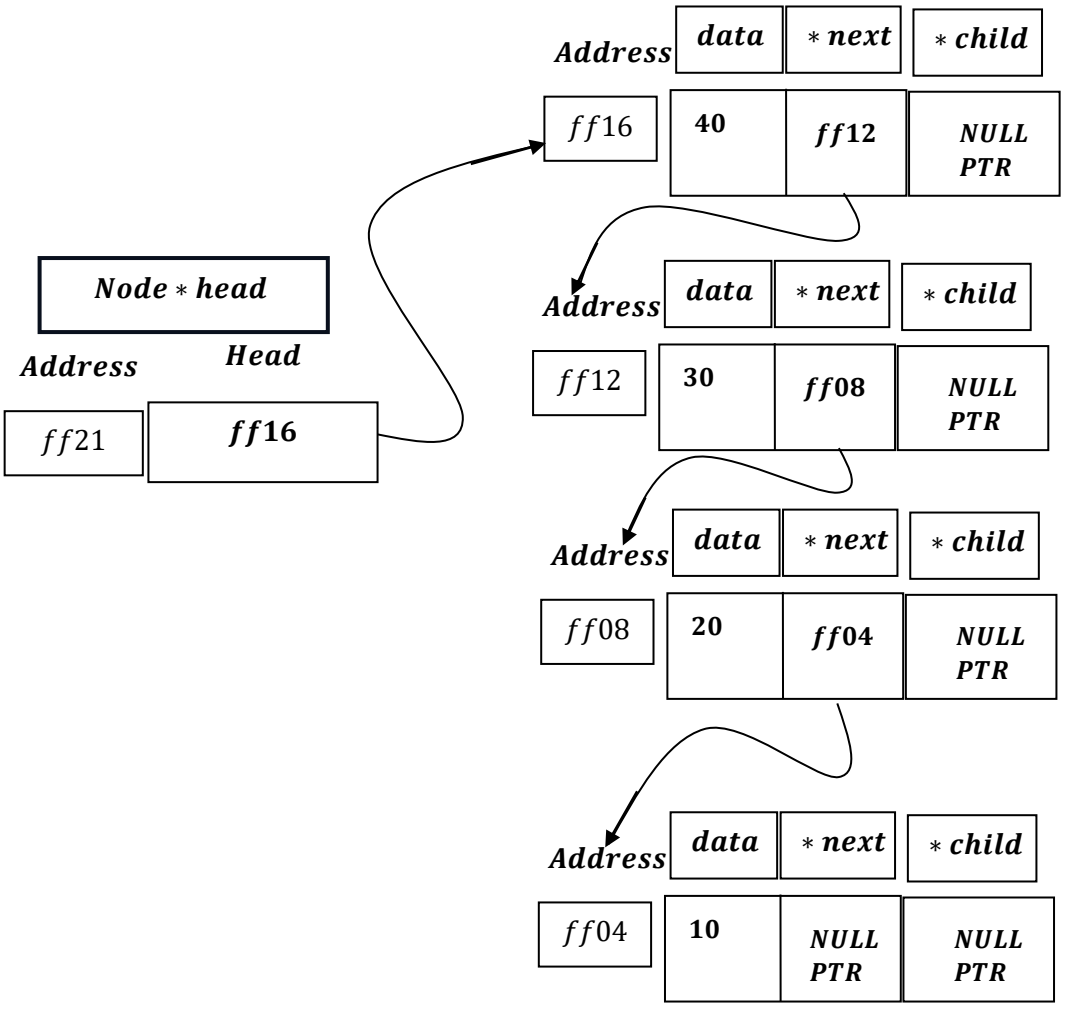
It will free the allocated memory by pointer variable newNode.



After , the , function gets over:



And , we are left with:


Address
data
* next
* child
ff16
40
ff12
NULL PTR
Node * head
Address
Head
ff21
ff16
Address
data
* next
* child
ff12
30
ff08
NULL PTR
Address
data
* next
* child
ff08
20
ff04
NULL PTR
Address
data
* next
* child
ff04
10
NULL PTR
NULL PTR